Business Intelligence
Mini Project Report
On
Churn Prediction Using Machine Learning
(A.Y 2024-2025)

PREPARED BY

Name	ID
Anand Dane	VU4F2223094
Swaroop Sawant	VU4F2223100
Prathamesh Thakare	VU4F2223103
Omkar Patil	VU4F2223108

Class : TE IT-B (Batch -B)

Guided By : Prof. Sachin Barade

“Churn Prediction Using Machine Learning”

Problem Statement:

In highly competitive industries such as telecommunications, finance, and e-commerce, customer retention is as critical as customer acquisition. However, organizations often face the challenge of customer churn, where users discontinue their service, resulting in significant revenue losses and increased marketing costs to acquire new customers.

Traditional methods of identifying churn are reactive and heavily reliant on manual reviews or basic statistical analysis, which fail to capture complex patterns in customer behavior. Without timely and accurate predictions, companies struggle to intervene effectively and reduce churn rates.

This project addresses the need for a proactive, data-driven Cyber Threat Intelligence and Churn Prediction System using machine learning algorithms. The system leverages historical customer data, including demographic details, service usage patterns, billing information, and tenure, to accurately predict whether a customer is likely to churn.

The solution involves collecting and preprocessing the data by handling missing values, encoding categorical variables, and scaling features. Important factors such as monthly charges, tenure, service type, and support history are analyzed to uncover their impact on customer churn. The project uses machine learning models like Random Forest Classifier and XGBoost to train predictive models that classify churn with high accuracy. The performance of these models is evaluated using metrics such as accuracy, precision, recall, and ROC-AUC scores.

Visualizations, including feature importance graphs, churn distributions, and correlation heatmaps, help in interpreting the model’s output and generating actionable insights. By implementing this system, businesses can gain a deeper understanding of customer behavior and apply targeted strategies to reduce churn, enhance customer satisfaction, and ensure long-term profitability.

About the dataset :

This dataset contains information about **7,043 customers** from what appears to be a telecom company. It seems designed to analyze or predict **customer churn**—that is, whether a customer will stop using the service..

The dataset contains various types of information about each customer. It includes personal details such as gender, whether they are a senior citizen, and whether they have a partner or any dependents. It also provides service-related information, like whether the customer has phone service, internet access, streaming services, or tech support.

Additionally, it includes account-related data, such as the type of contract the customer has, the method of payment they use, their monthly charges, and the total charges incurred. Finally, there is a churn column that indicates whether or not the customer has left the company, which is likely the main focus of analysis or prediction.

Preprocessing:

The preprocessing phase in this project involves several essential steps to prepare the dataset for machine learning. First, the dataset is loaded, and missing values are checked to ensure data quality. Categorical columns are then encoded using LabelEncoder to convert text labels into numerical values suitable for model input. The data is split into features (X) and the target variable (y), which represents customer churn. To ensure consistency in feature scaling and to improve model performance, the features are standardized using StandardScaler. This transforms the data to have a mean of zero and a standard deviation of one. Finally, the processed data is divided into training and testing sets using train_test_split, allowing the model to be trained on one portion of the data and evaluated on another to assess its generalization performance.

Algorithms used:

Random Forest Regressor (Supervised Learning):

The Random Forest Classifier is an ensemble learning algorithm that operates by constructing multiple decision trees during training and combining their results to improve accuracy and reduce overfitting. It works by selecting random subsets of the data and features to build each tree, ensuring that the model is robust and less prone to being overly influenced by noisy data or outliers. Random Forest is particularly well-suited for churn prediction because it can handle a mix of categorical and numerical variables and provides insight into which features are most important for making predictions. In your analysis, it performed with solid accuracy and highlighted key factors such as contract type, monthly charges, and customer tenure as significant indicators of churn behavior**.**

XGBoost Regressor (Supervised Learning):

XGBoost Classifier (Extreme Gradient Boosting) is a more advanced and powerful machine learning algorithm that builds decision trees sequentially, with each new tree attempting to correct the errors made by the previous ones. It is known for its high performance, efficiency, and scalability, making it a popular choice for predictive modeling tasks, including customer churn. XGBoost includes built-in regularization, which helps reduce overfitting and improves generalization.
In your project, XGBoost performed similarly to Random Forest in terms of accuracy but identified the contract type as the most critical feature by a large margin, suggesting that the nature of the customer's agreement strongly influences their likelihood to leave. This level of precision makes XGBoost especially valuable for uncovering subtle patterns and trends in complex datasets**.**

Information about ROC Curve:

The ROC (Receiver Operating Characteristic) curve and the Precision-Recall curve are key tools for evaluating the performance of classification models. The ROC curve plots the true positive rate against the false positive rate at various threshold levels, with the Area Under the Curve (AUC) indicating the model's ability to distinguish between classes—where an AUC of 1 represents a perfect model and 0.5 indicates random guessing. It is most useful for balanced datasets. In contrast, the Precision-Recall curve focuses on the trade-off between precision (how many predicted positives are actually positive) and recall (how many actual positives are correctly identified), making it especially valuable for imbalanced datasets where the positive class is rare. Together, these curves provide comprehensive insight into a model's classification performance, helping choose the best threshold and compare different models effectively.

Information about Recall Curve:

The Recall Curve is a visualization that helps evaluate how well a classification model identifies positive instances across different threshold levels. Recall, also known as sensitivity or true positive rate, measures the proportion of actual positives that are correctly predicted by the model. A Recall Curve typically appears in the context of a Precision-Recall Curve, where it shows how recall varies with precision. High recall indicates that the model successfully captures most of the positive cases, which is particularly important in scenarios like medical diagnosis or fraud detection where missing a positive instance could have serious consequences. However, improving recall often comes at the cost of precision, meaning the model may also predict more false positives. The curve helps in understanding this trade-off and selecting the most appropriate threshold for optimal model performance.

Loading Dataset

```
# 📥 Load dataset
df = pd.read_csv("my_dataset.csv")

display(Markdown("## 🔍 Dataset Preview"))
display(df.head())
```

Import Necessary Libraries

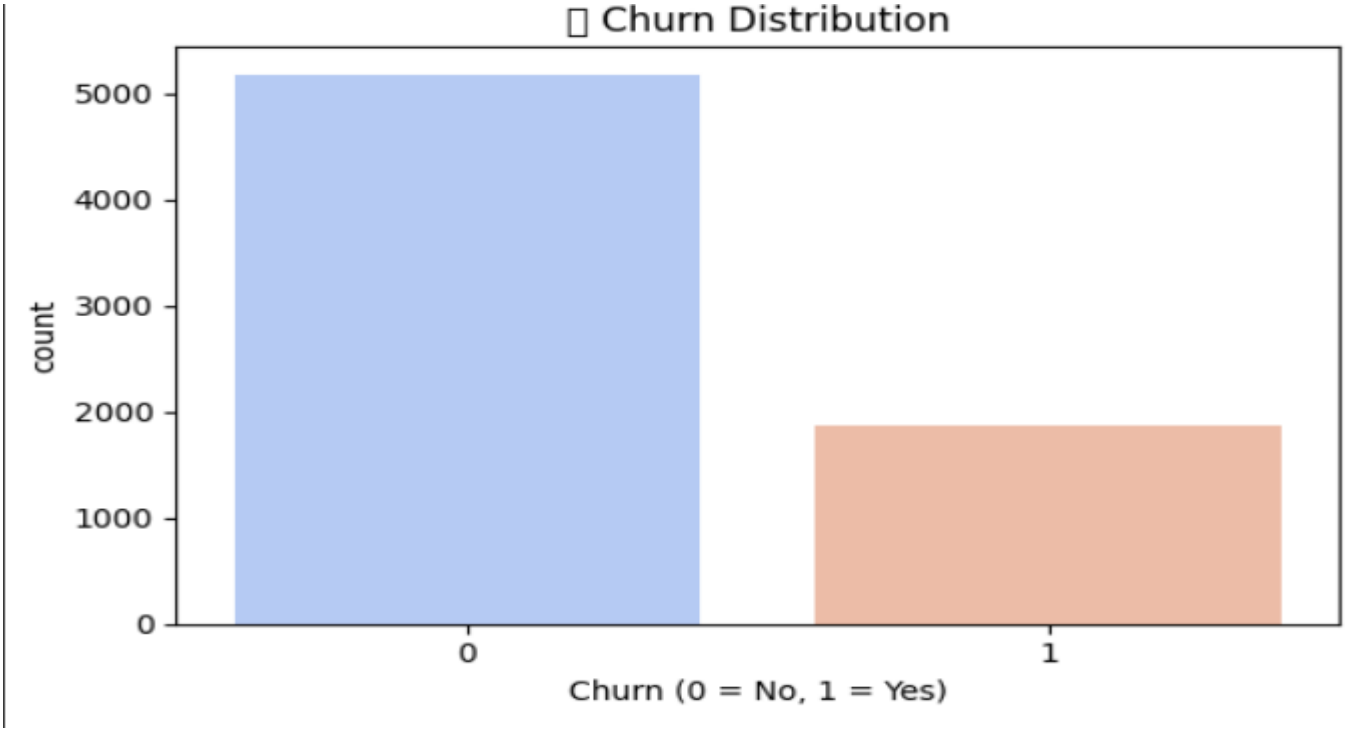
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Reading Data into Pandas

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	Strea
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	

5 rows × 21 columns

Data Visualization



Data Preprocessing

```
# Load Dataset
file_path = "my_dataset.csv"  # Ensure your dataset is in CSV format
churn_df = pd.read_csv(file_path)

# Display first few rows
print(churn_df.head())

# Check for missing values
print("Missing Values:\n", churn_df.isnull().sum())

# Drop unnecessary columns (if any, based on dataset exploration)
# churn_df = churn_df.drop(['CustomerID'], axis=1)  # Uncomment if needed
```

```
# Encode categorical variables
label_encoders = {}
for col in churn_df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    churn_df[col] = le.fit_transform(churn_df[col])
    label_encoders[col] = le

# Split dataset into features (X) and target variable (y)
X = churn_df.drop(columns=['Churn'])  # 'Churn' should be the target column
y = churn_df['Churn']
```

```
# Normalize numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Train a Classification Model (Random Forest)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

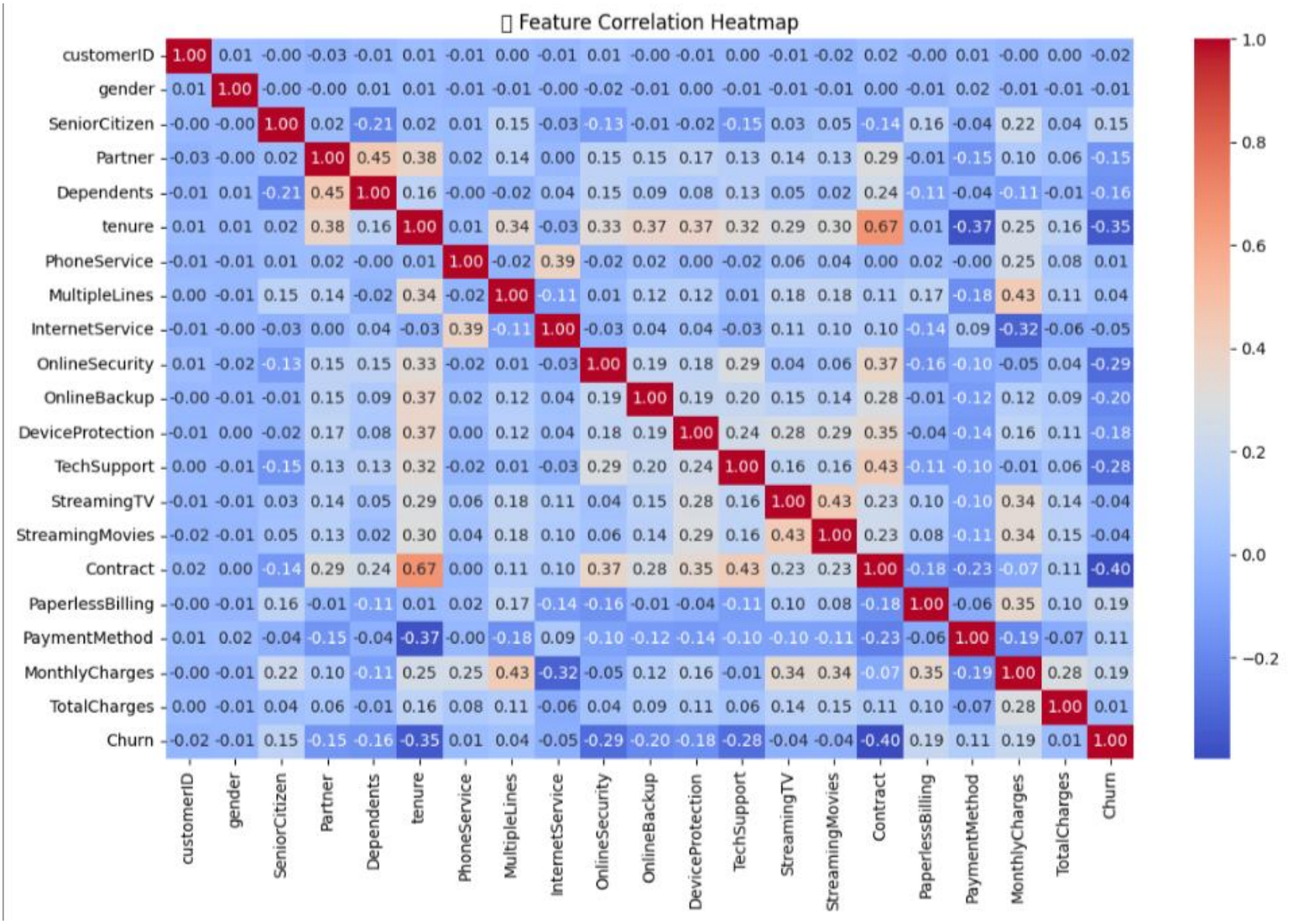
Correlation Heatmap

Input

```
# Correlation Heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("📊 Feature Correlation Heatmap")
plt.tight_layout()
plt.show()


from sklearn.metrics import precision_recall_curve, roc_curve, auc
```

Output



ROC & Precision- Recall Curve:

Input:

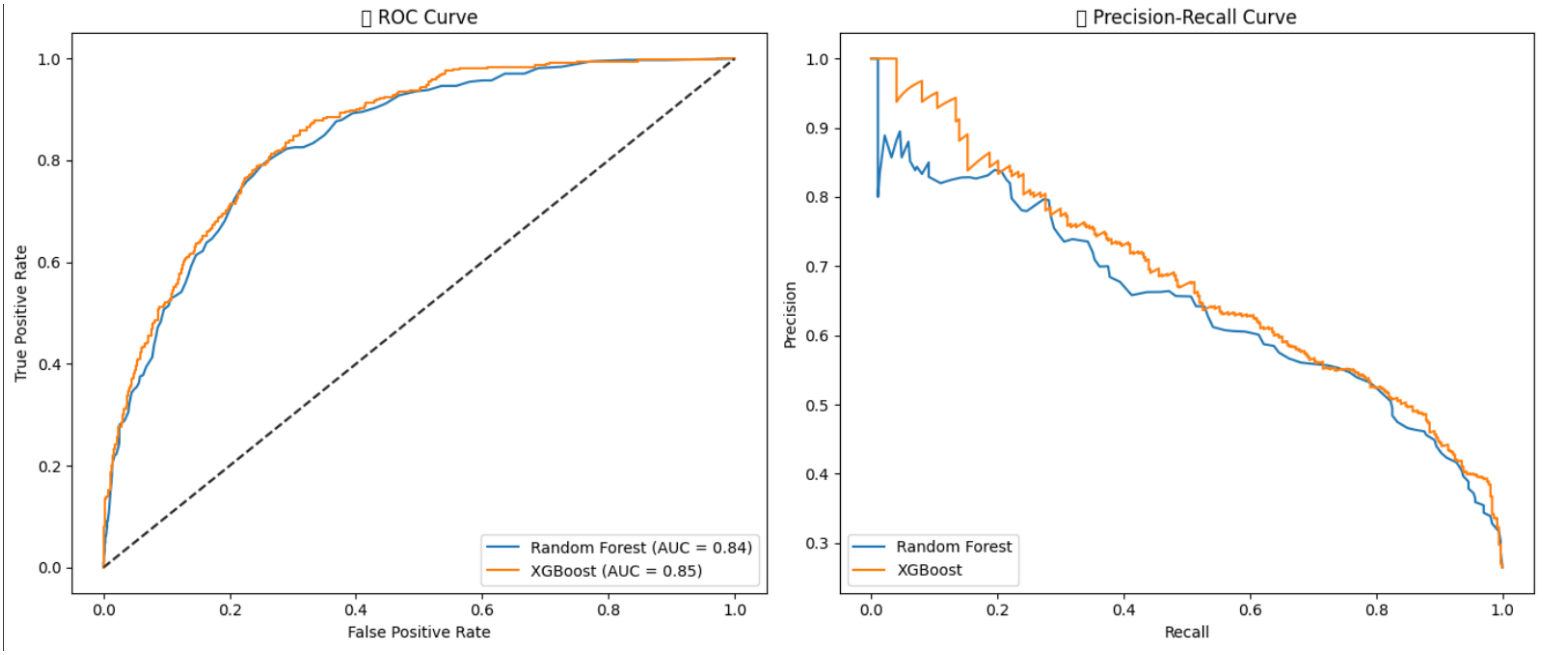
```
    # ROC Curve
    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_auc = auc(fpr, tpr)

    plt.subplot(1, 2, 1)
    plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc:.2f})")
    plt.plot([0, 1], [0, 1], 'k--', alpha=0.6)
    plt.title("🔵 ROC Curve")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.legend(loc='lower right')

    # Precision-Recall Curve
    precision, recall, _ = precision_recall_curve(y_test, y_score)
    plt.subplot(1, 2, 2)
    plt.plot(recall, precision, label=f"{name}")
    plt.title("🟣 Precision-Recall Curve")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.legend(loc='lower left')

plt.tight_layout()
plt.show()
```

Output:



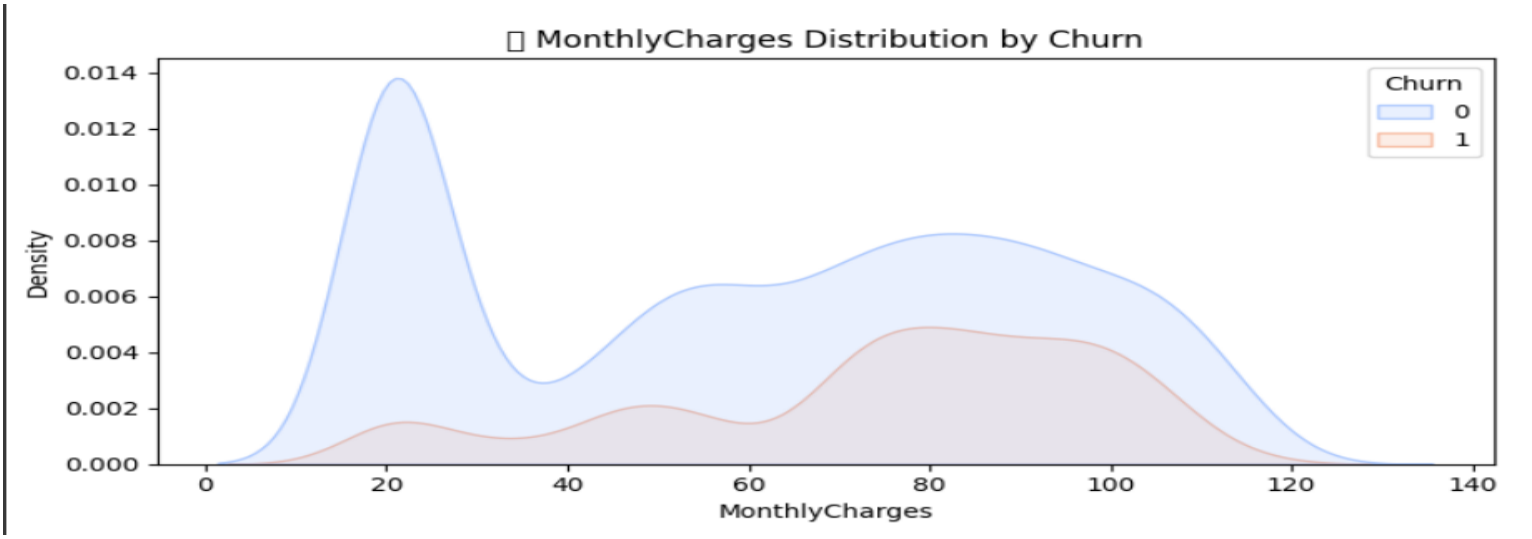
Churn vs Monthly Charges Distribution

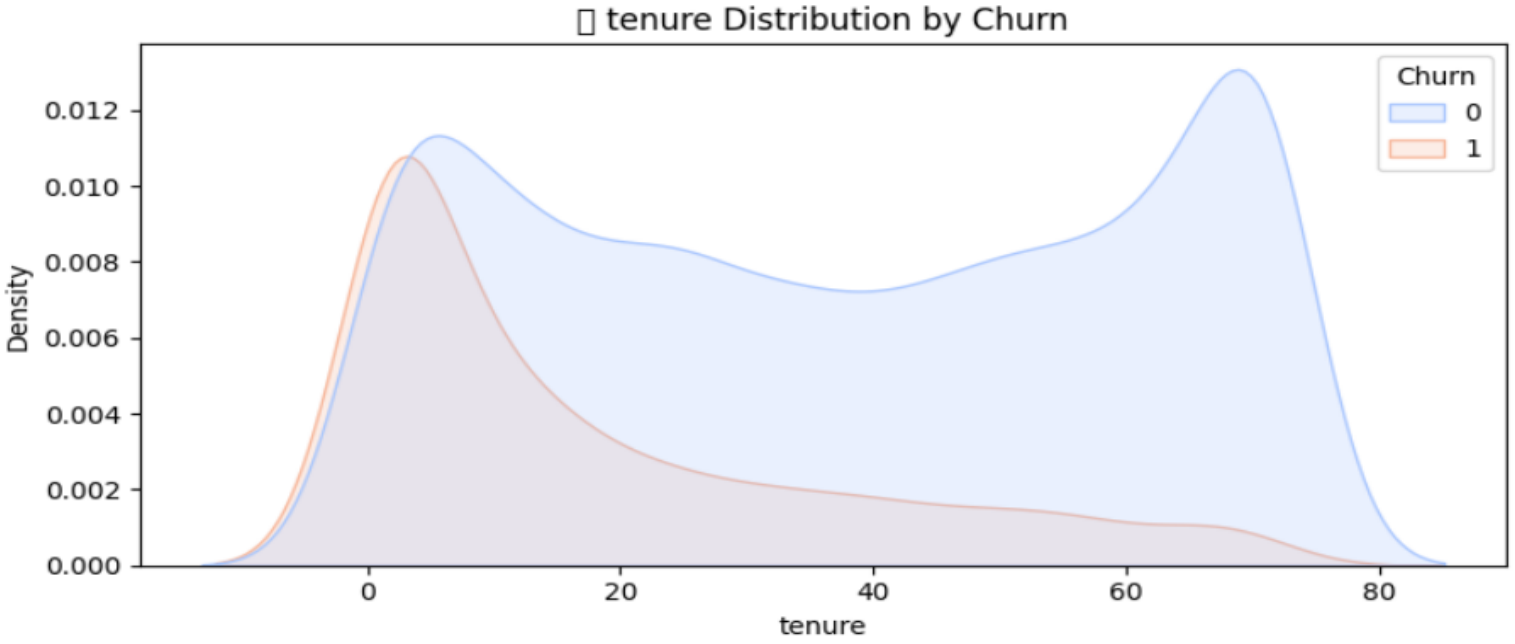
Input:

```
# 📉 Churn vs Monthly Charges or Tenure (Example)
top_features = ['MonthlyCharges', 'tenure'] if 'MonthlyCharges' in df.columns and 'tenure' in df.columns else df.columns[:2]

for feature in top_features:
    plt.figure(figsize=(8, 4))
    sns.kdeplot(data=df, x=feature, hue="Churn", fill=True, palette='coolwarm')
    plt.title(f"📈 {feature} Distribution by Churn")
    plt.tight_layout()
    plt.show()
```

Output





Train Random Forest & XGBoost Models

```
# 🧠 Train Models
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(n_estimators=100, learning_rate=0.1, use_label_encoder=False, eval_metric='logloss', random_state=42)
}
```

Output of Random Forest & XGBoost Models

Output of Random forest:

🔎 **Model: Random Forest**

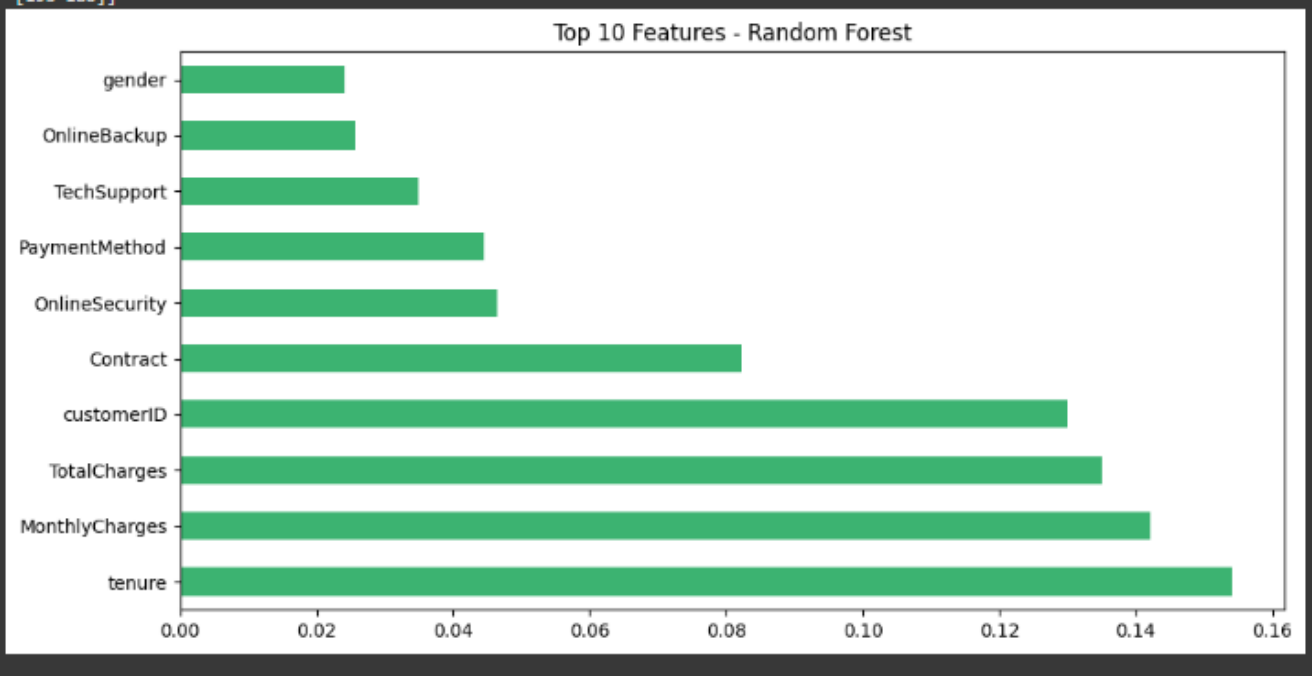
✅ **Accuracy: 0.7963**

```
Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.91      0.87      1036
           1       0.66      0.48      0.56       373

    accuracy                           0.80      1409
   macro avg       0.74      0.70      0.71      1409
weighted avg       0.78      0.80      0.79      1409

Confusion Matrix:
 [[942  94]
 [193 180]]
```



Output of XGBoost:

Model: XGBoost

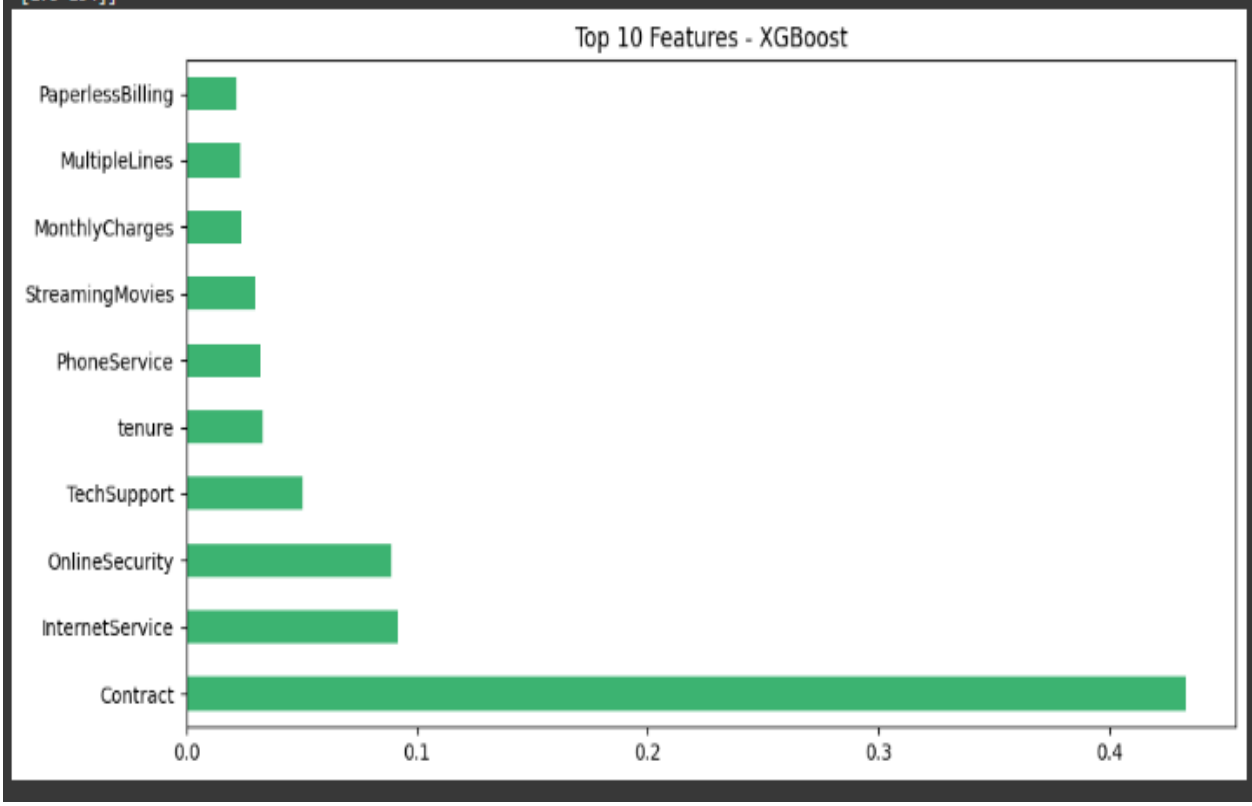
Accuracy: 0.7984

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.90	0.87	1036
1	0.65	0.52	0.58	373
accuracy			0.80	1409
macro avg	0.74	0.71	0.72	1409
weighted avg	0.79	0.80	0.79	1409

Confusion Matrix:

```
[[931 105]
 [179 194]]
```



Observations:

- The XGBoost model provided the most accurate Remaining Useful Life (RUL) predictions among all models tested.
- Data preprocessing, including outlier removal and Min-Max scaling, significantly improved model performance and stability.
- Feature selection helped reduce noise by focusing only on the most informative sensor readings.
- KMeans clustering revealed distinct patterns in sensor behavior under different operating conditions, aiding in failure analysis.

Conclusion:

This project successfully implemented a machine learning-based system to predict customer churn using telecom data. After preprocessing and analyzing key features like contract type, monthly charges, and tenure, models like Random Forest and XGBoost were trained. XGBoost delivered the highest accuracy, making it the most effective model. The system helps businesses identify at-risk customers early and apply targeted strategies to reduce churn and improve customer retention.